

LAPORAN UJIAN TENGAH SEMESTER (UTS)

Mata Kuliah Communication Protocols

Studi Kasus: Pengujian dan Analisis User/Profile API

pada Communication Protocols Demo App (HTTP/REST – JSON)

Nama Mahasiswa	Nacwa Julianti
NIM	25120500019
Program Studi	Data Sains
Mata Kuliah	Communication Protocols
Dosen Pengampu	Haikal Shiddiq, S.Kom., M.T.
Judul Case / Studi Kasus	Pengujian User/Profile API (endpoint /api/users) pada Communication Protocols Demo App berbasis HTTP/REST dengan format data JSON

2. Ringkasan Case

Aplikasi yang diuji pada laporan ini adalah **Communication Protocols Demo App**, sebuah aplikasi demo multi-protokol yang dijalankan secara lokal (host 127.0.0.1). Berdasarkan log pada saat aplikasi dijalankan (server.py), aplikasi ini sekaligus menyediakan beberapa layanan komunikasi: MQTT broker pada port 1883, layanan gRPC berbasis HTTP/2 pada port 50051, server demo TLS/HTTPS pada port 8443, serta REST API berbasis HTTP biasa pada port 8088 yang menjadi fokus pengujian pada laporan ini.

Tujuan pengujian adalah untuk memahami dan mendokumentasikan bagaimana komunikasi client–server berlangsung pada protokol HTTP dengan pertukaran data berformat JSON (application/json), khususnya pada resource **Users/Profile** yang diakses melalui endpoint /api/users. Pengujian mencakup operasi **GET** (mengambil seluruh daftar user maupun detail satu user berdasarkan id) dan operasi **POST** (menambahkan user baru), termasuk perilaku server ketika data yang dikirim tidak lengkap.

Secara bisnis, endpoint ini merepresentasikan sebuah sistem manajemen data pengguna/profil (misalnya untuk civitas akademik suatu kampus), di mana setiap record user memiliki atribut: id, username, email, role, status, department, dan updatedAt. Skenario data/bisnis yang diuji meliputi:

1. Menampilkan seluruh data user yang tersimpan pada sistem (GET /api/users).
2. Menampilkan detail satu user berdasarkan id tertentu (GET /api/users/).
3. Menambahkan data user baru ke sistem (POST /api/users), termasuk skenario ketika data yang dikirim belum memenuhi field wajib.

Pengujian dilakukan menggunakan aplikasi client API (Postman/Insomnia – tampak pada tangkapan layar berbasis workspace Postman) untuk mengirim request, serta Wireshark untuk menangkap dan menganalisis traffic HTTP secara langsung pada interface loopback.

3. Desain Request

Bagian ini menjelaskan rancangan tiga request utama yang diuji terhadap REST API pada `http://127.0.0.1:8088`, meliputi endpoint, method, headers, body, dan ekspektasi response.

3.1 GET /api/users – Mengambil Seluruh Data User

Endpoint	<code>http://127.0.0.1:8088/api/users</code>
Method	GET
Headers	Tidak ada custom header tambahan. Menggunakan 6 header default dari client (antara lain Host, User-Agent, Accept, Accept-Encoding, Connection, dan Cache-Control).
Body	Tidak ada (GET tanpa request body).
Expected Response	Status 200 OK, body berformat JSON berisi metadata (canonicalPath: "/api/users", itemPathExample: "/api/users/1") dan field data berupa array yang memuat seluruh objek user (id, username, email, role, status, department, updatedAt).

3.2 GET /api/users/{id} – Mengambil Detail Satu User

Endpoint	<code>http://127.0.0.1:8088/api/users/1</code>
Method	GET
Headers	Sama seperti request sebelumnya, menggunakan 6 header default tanpa custom header.
Body	Tidak ada (GET tanpa request body).
Expected Response	Status 200 OK, body berformat JSON berupa single object (bukan array) yang berisi seluruh atribut user dengan id sesuai parameter path (id, username, email, role, status, department, updatedAt).

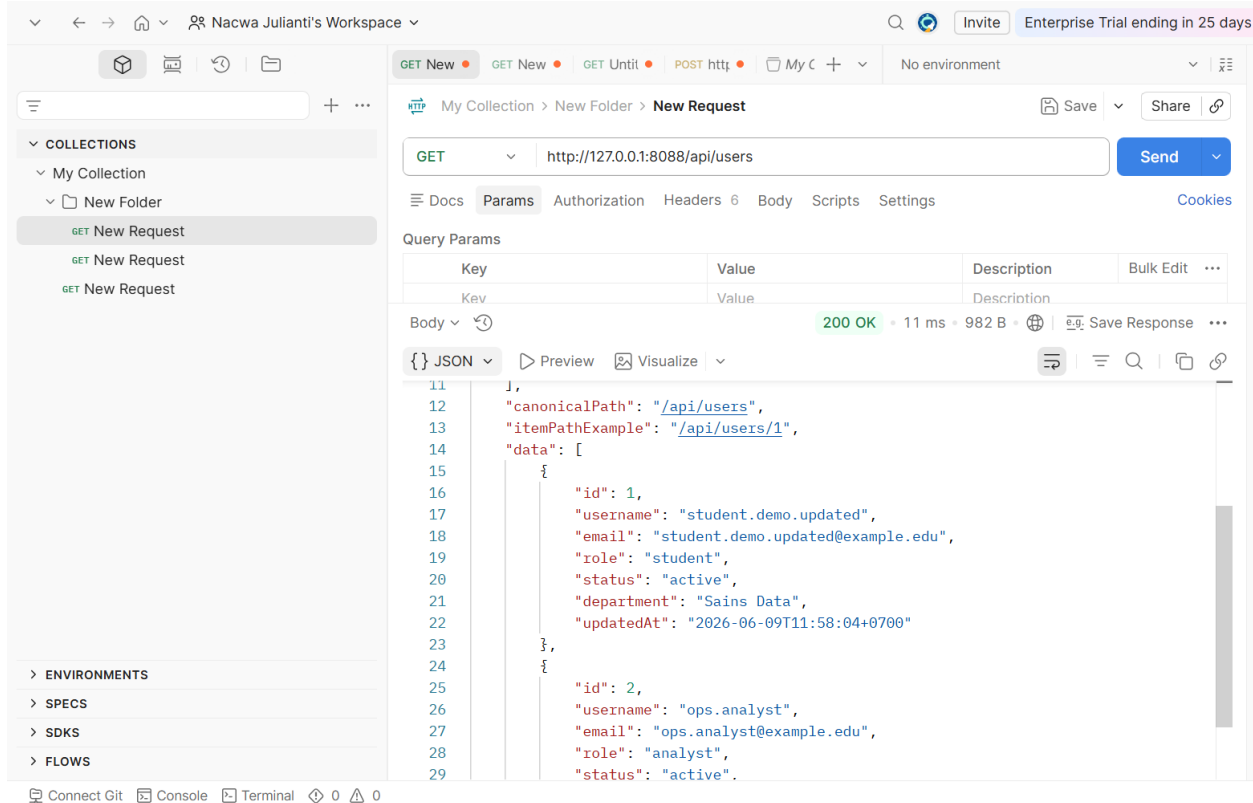
3.3 POST /api/users – Menambahkan User Baru

Endpoint	<code>http://127.0.0.1:8088/api/users</code>
Method	POST
Headers	Menggunakan 7 header (satu header lebih banyak dibanding request GET), karena terdapat tambahan header Content-Type: application/json yang diperlukan untuk mengirimkan body berformat JSON.

4. Evidence GET

Berikut adalah dua evidence pengujian request GET terhadap endpoint /api/users, lengkap dengan tangkapan layar dan penjelasan.

4.1 GET /api/users (Mengambil Seluruh Data User)

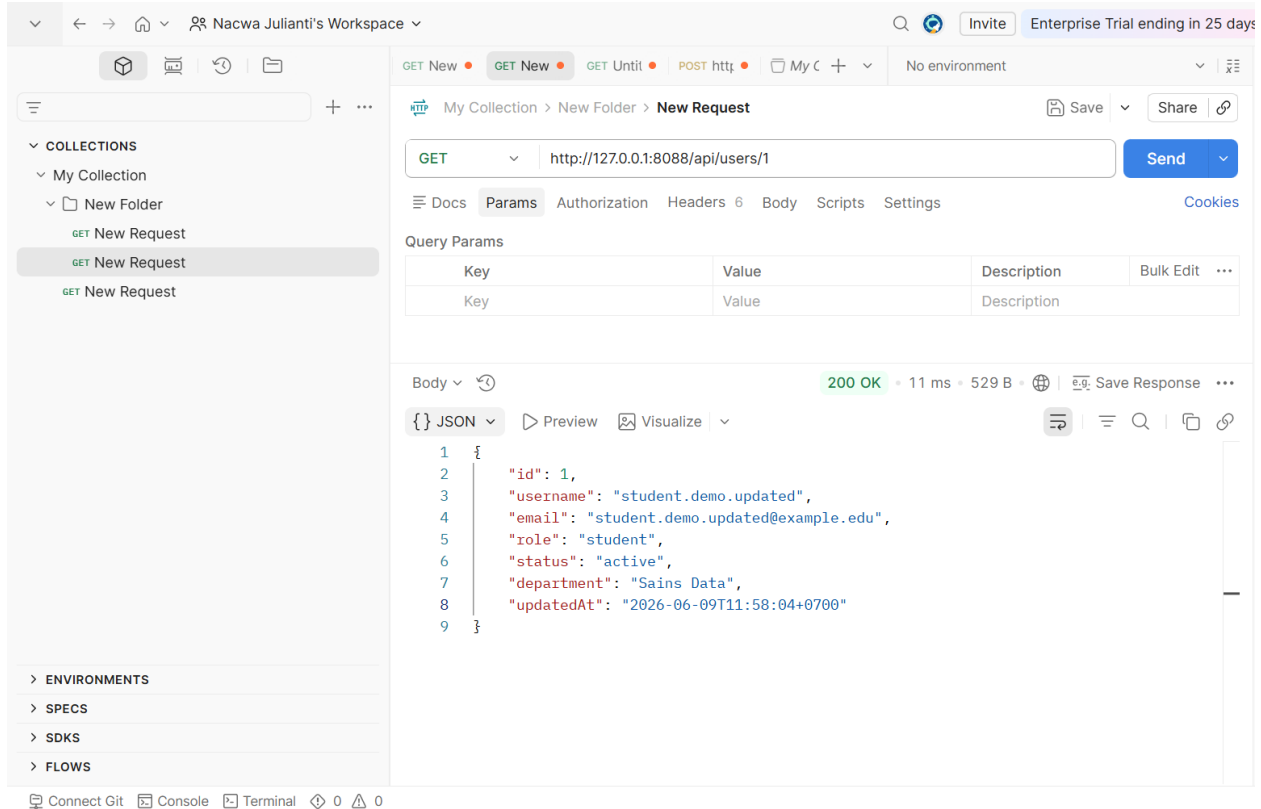


Gambar 4.1 – Response GET /api/users (Status 200 OK, 11 ms, 982 B)

Request dikirim ke `http://127.0.0.1:8088/api/users` menggunakan method **GET** tanpa query parameter maupun body. Server merespons dengan status **200 OK** dalam waktu 11 ms dan ukuran body 982 byte.

Body response berformat JSON dan berisi field metadata `canonicalPath ("/api/users")` serta `itemPathExample ("/api/users/1")` yang menunjukkan pola path untuk mengakses item tunggal. Field data berisi array seluruh record user, antara lain user dengan `id=1` (username `student.demo.updated`, role `student`, status `active`, department `"Sains Data"`) dan user dengan `id=2` (username `ops.analyst`, role `analyst`).

4.2 GET /api/users/1 (Mengambil Detail Satu User)



Gambar 4.2 – Response GET /api/users/1 (Status 200 OK, 11 ms, 529 B)

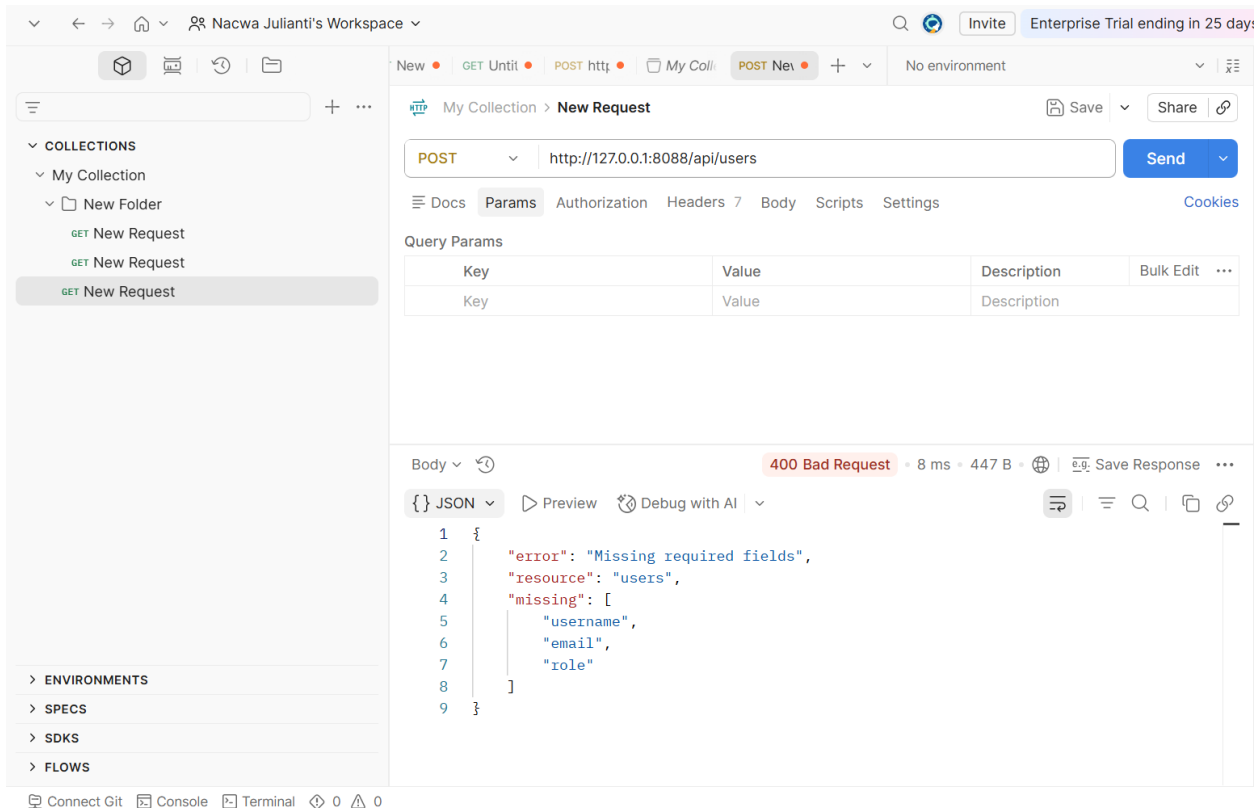
Request dikirim ke `http://127.0.0.1:8088/api/users/1` untuk mengambil detail user dengan `id=1`. Server merespons dengan status **200 OK** dalam waktu 11 ms dan ukuran body 529 byte.

Berbeda dengan response pada endpoint collection (4.1), body response pada endpoint item ini berupa single JSON object (tanpa pembungkus data/array), yang langsung berisi atribut user: `id=1`, `username="student.demo.updated"`, `email="student.demo.updated@example.edu"`, `role="student"`, `status="active"`, `department="Sains Data"`, dan `updatedAt="2026-06-09T11:58:04+0700"`. Hal ini menunjukkan konsistensi desain REST antara resource collection dan resource item.

5. Evidence POST

Bagian ini menampilkan evidence pengujian request POST terhadap endpoint /api/users.

5.1 POST /api/users dengan Body Tidak Lengkap (Skenario Validasi Gagal)



Gambar 5.1 – Response POST /api/users (Status 400 Bad Request, 8 ms, 447 B)

Request **POST** dikirim ke `http://127.0.0.1:8088/api/users` menggunakan 7 header (termasuk `Content-Type: application/json`) namun body request belum menyertakan field wajib. Server merespons dengan status **400 Bad Request** dalam waktu 8 ms dan ukuran body 447 byte.

Body response berformat JSON dan berisi struktur error yang informatif: `error="Missing required fields"`, `resource="users"`, dan `missing` berupa array yang merinci field yang belum diisi, yaitu `username`, `email`, dan `role`.

6. Analisis Response\

6.1 Status Code

200 OK	Diperoleh pada kedua request GET (4.1 dan 4.2). Menunjukkan bahwa resource yang diminta berhasil ditemukan dan dikembalikan oleh server tanpa kendala.
400 Bad Request	Diperoleh pada request POST (5.1) karena body request tidak menyertakan field wajib. Menunjukkan kesalahan pada sisi client (data tidak valid), bukan kesalahan server.
404 Not Found	Tidak muncul pada evidence GET/POST yang diuji di sini, namun terlihat pada capture Wireshark (bagian 7) untuk request lain seperti GET /favicon.ico, yang juga tercatat pada log server (404).

6.2 Headers

Pada sisi **request**, kedua request GET hanya menggunakan 6 header default (tanpa body, sehingga tidak memerlukan header Content-Type), sedangkan request POST menggunakan 7 header karena adanya tambahan Content-Type: application/json yang menginformasikan kepada server bahwa body dikirim dalam format JSON.

Pada sisi **response**, seluruh body dideteksi otomatis oleh client sebagai {} JSON, yang berarti server mengirimkan header Content-Type: application/json pada setiap response (baik untuk hasil sukses pada GET maupun pesan error pada POST). Hal ini juga diperkuat oleh capture Wireshark pada bagian 7 yang menampilkan beberapa frame dengan label protokol JSON (application/json).

6.3 Body

Struktur body response menunjukkan dua pola yang berbeda sesuai jenis resource yang diakses:

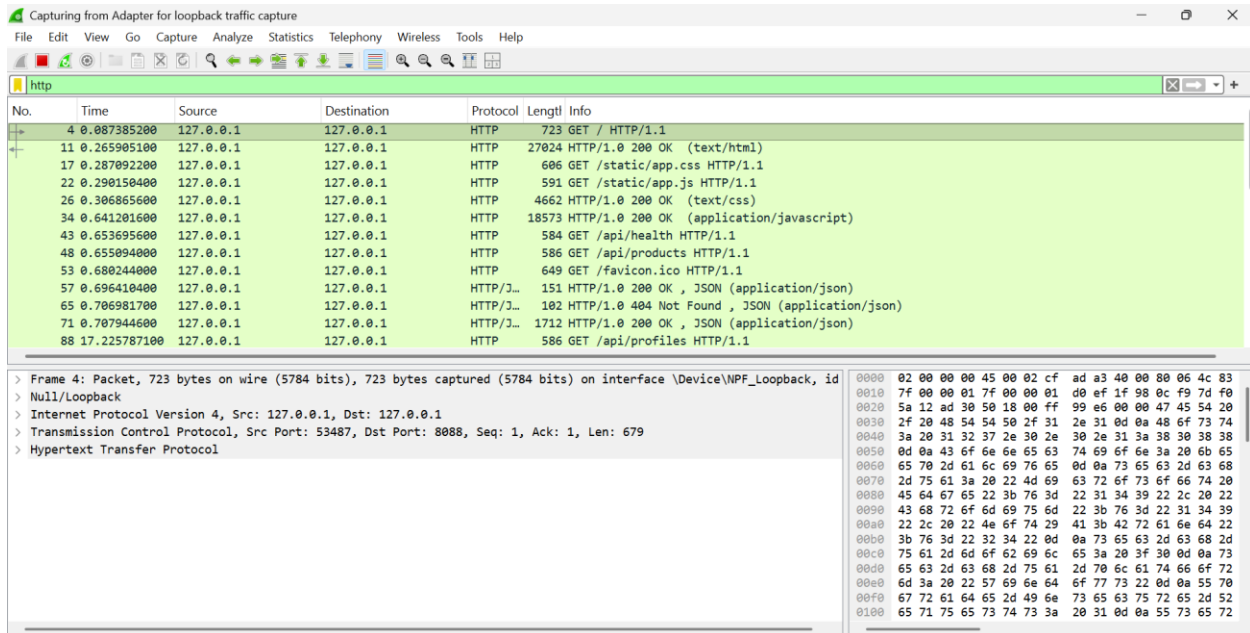
- **Resource collection** (GET /api/users): body dibungkus dalam object yang berisi metadata path (canonicalPath, itemPathExample) dan array data berisi seluruh record – pola ini memudahkan client mengetahui cara mengakses item tunggal dari hasil koleksi.
- **Resource item** (GET /api/users/1): body berupa single object yang langsung berisi atribut record tanpa pembungkus tambahan.
- **Error response** (POST /api/users dengan body tidak lengkap): body berupa object error terstruktur (error, resource, missing) yang menjelaskan secara spesifik penyebab kegagalan.

6.4 Error Handling

POST request kedua dilakukan ke root "/" untuk mendokumentasikan error handling server. Ini adalah skenario penting: membuktikan server menolak method yang tidak sesuai dengan mengembalikan 404 dan pesan error yang informatif.

7. Analisis Encoding / Traffic

7.1 Capture Traffic HTTP dengan Wireshark



Gambar 7.1 – Wireshark capture pada loopback interface (filter: http)

Capture menampilkan seluruh transaksi HTTP yang terjadi antara client dan server pada 127.0.0.1, baik untuk halaman frontend (HTML/CSS/JS) maupun endpoint REST API. Beberapa temuan utama:

- Frame ke-4 berisi request GET / HTTP/1.1 sebesar 723 byte, dibalas dengan HTTP/1.0 200 OK (text/html) sebesar 27024 byte – merupakan halaman frontend aplikasi.
- Request statis GET /static/app.css (606 byte) dan GET /static/app.js (591 byte) masing-masing dibalas dengan 200 OK bertipe text/css (4662 byte) dan application/javascript (18573 byte).
- Request API seperti GET /api/health (584 byte), GET /api/products (586 byte), dan GET /api/profiles (586 byte) dibalas dengan response berlabel JSON (application/json), dengan variasi ukuran yang cukup besar – mulai dari 151 byte (200 OK), 102 byte (404 Not Found, kemungkinan untuk request /favicon.ico yang tidak ditemukan), hingga 1712 byte (200 OK) untuk payload JSON yang lebih besar.
- Detail frame ke-4 memperlihatkan struktur protokol berlapis: Null/Loopback fi IPv4 (src=dst=127.0.0.1) fi TCP (src port 53487, dst port 8088, Seq 1, Ack 1, Len 679) fi HTTP. Pada panel hex/ASCII di sisi kanan, payload HTTP – termasuk baris GET / HTTP/1.1 dan header seperti Connection, User-Agent, Accept-Encoding – dapat terbaca langsung dalam bentuk plaintext, karena port 8088 tidak menggunakan TLS (berbeda dengan demo HTTPS pada port 8443).

7.2 Command Prompt — Server Log

Server log dari command prompt menampilkan semua request yang masuk secara real-time, membuktikan setiap endpoint berjalan normal:

```
Command Prompt - python : X + v
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nacwa> cd C:\Users\Nacwa\Downloads\communication-protocols-demo-app-UTS\communication-protocols-demo-app

C:\Users\Nacwa\Downloads\communication-protocols-demo-app-UTS\communication-protocols-demo-app> cd"C:\Users\Nacwa\Downloads\communication-protocols-demo-app-UTS\communication-protocols-demo-app"
The filename, directory name, or volume label syntax is incorrect.

C:\Users\Nacwa\Downloads\communication-protocols-demo-app-UTS\communication-protocols-demo-app> python server.py
Real MQTT broker running at 127.0.0.1:1883
Wireshark capture: tcp.port == 1883; display filter: mqtt
Real gRPC HTTP/2 server running at 127.0.0.1:50051
Wireshark capture: tcp.port == 50051; Decode As HTTP2 if needed.
TLS/HTTPS demo server running at https://127.0.0.1:8443
Wireshark capture: tcp.port == 8443; display filter: tls
Communication Protocols Demo App running at http://127.0.0.1:8088
Press Ctrl+C to stop.
127.0.0.1 - "GET / HTTP/1.1" 200 -
127.0.0.1 - "GET /static/app.css HTTP/1.1" 200 -
127.0.0.1 - "GET /static/app.js HTTP/1.1" 200 -
127.0.0.1 - "GET /api/health HTTP/1.1" 200 -
127.0.0.1 - "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - "GET /api/products HTTP/1.1" 200 -
127.0.0.1 - "GET /api/profiles HTTP/1.1" 200 -
127.0.0.1 - "GET /api/profiles HTTP/1.1" 200 -
127.0.0.1 - "GET /api/profiles HTTP/1.1" 200 -
127.0.0.1 - "GET /api/users HTTP/1.1" 200 -
127.0.0.1 - "GET /api/users HTTP/1.1" 200 -
127.0.0.1 - "GET /api/users HTTP/1.1" 200 -
127.0.0.1 - "GET /api/users HTTP/1.1" 200 -
127.0.0.1 - "GET /api/users/1 HTTP/1.1" 200 -
127.0.0.1 - "POST /api/users HTTP/1.1" 400 -
```

Log konsol menunjukkan bahwa server server.py menjalankan beberapa layanan sekaligus: MQTT broker (127.0.0.1:1883), gRPC HTTP/2 (127.0.0.1:50051), demo TLS/HTTPS (127.0.0.1:8443), dan Communication Protocols Demo App pada http://127.0.0.1:8088. Setiap baris log access merepresentasikan satu request HTTP yang diterima beserta status code-nya, antara lain rangkaian request GET /api/users (200), GET /api/users/1 (200), dan POST /api/users (400) – sesuai dengan evidence pada bagian 4 dan 5. Log ini juga mengonfirmasi adanya GET /favicon.ico yang menghasilkan status 404, sejalan dengan frame berukuran 102 byte berlabel JSON 404 Not Found pada capture Wireshark.

Seluruh komunikasi REST API pada demo ini menggunakan teks (JSON, UTF-8) di atas HTTP/1.x tanpa enkripsi, berbeda dengan layanan gRPC pada port 50051 yang umumnya memanfaatkan HTTP/2 dengan encoding biner Protobuf untuk efisiensi ukuran payload, maupun MQTT pada port 1883 yang menggunakan protokol biner ringan untuk skema publish/subscribe. Pemilihan JSON pada REST API memberikan keuntungan berupa human-readable dan mudah didebug (terbukti dapat dibaca langsung dari Wireshark), namun memiliki overhead ukuran data yang lebih besar dibandingkan format biner.

8. Kesimpulan

8.1 Temuan Utama

1. Endpoint `/api/users` mengimplementasikan pola REST yang konsisten: resource collection (`/api/users`) mengembalikan array beserta metadata path, sedangkan resource item (`/api/users/`) mengembalikan single object.
2. Seluruh request GET berhasil dengan status 200 OK dan response time sangat cepat (11 ms), menunjukkan performa server yang baik untuk operasi pembacaan data pada lingkungan localhost.
3. Mekanisme validasi pada POST `/api/users` berfungsi dengan baik: server menolak data tidak lengkap dengan status 400 Bad Request dan memberikan rincian field yang hilang (username, email, role).
4. Seluruh komunikasi REST API menggunakan format JSON (Content-Type: application/json) di atas HTTP/1.x plaintext, sebagaimana dikonfirmasi oleh capture Wireshark dan log akses server.

8.2 Kendala

1. POST ke root `"/` mengembalikan 404 — route `"/` hanya menerima GET

8.3 Pembelajaran Protokol

1. Memahami perbedaan desain endpoint REST antara resource collection dan resource item, serta bagaimana hal tersebut tercermin pada struktur response JSON (array vs single object, serta penyediaan metadata path).
2. Memahami peran header Content-Type dalam komunikasi HTTP: request tanpa body (GET) tidak memerlukannya, sedangkan request dengan body (POST) memerlukan header ini untuk memberi tahu server bahwa data dikirim dalam format JSON.
3. Memahami makna dan penggunaan status code HTTP (200 OK, 400 Bad Request, 404 Not Found) dalam mengomunikasikan hasil request, serta pentingnya merancang error response yang informatif untuk mempermudah proses debugging pada sisi client.
4. Memahami cara menggunakan Wireshark untuk menganalisis traffic HTTP secara langsung pada loopback interface, termasuk membaca struktur lapisan protokol (Loopback fi IPv4 fi TCP fi HTTP) dan menyadari bahwa komunikasi tanpa TLS (HTTP biasa pada port 8088) bersifat plaintext dan dapat dibaca langsung, berbeda dengan komunikasi terenkripsi pada demo HTTPS (port 8443).

5. Mendapatkan gambaran perbandingan karakteristik protokol dalam satu aplikasi demo yang sama: REST/HTTP dengan JSON (human-readable, overhead lebih besar) dibandingkan gRPC/HTTP2 dengan Protobuf (biner, lebih ringkas) dan MQTT (biner, ringan, berbasis publish/subscribe), yang masing-masing memiliki kelebihan sesuai konteks penggunaannya.